

PortMira

A local-first portfolio mirror.

May 2026

4 Team Members

Python · Streamlit or Flask

Local-First · Privacy-Conscious

PortMira is a local-first personal finance dashboard built in Python that lets users input assets and liabilities, fetch live market prices through free APIs, and see their net worth and allocation clearly — without sending any data to the cloud.

MOTIVATION

Personal financial data is rarely in one place. Bank balances, brokerage holdings, crypto positions, loan balances, and cash sit across separate apps, websites, spreadsheets, and memory. Getting a complete, honest picture of one's financial position usually requires significant manual effort — or handing sensitive data to a commercial service with opaque privacy practices.

Most personal finance tools push users toward one of two extremes: a bloated banking app focused on spending categories, or an investment platform built around growth predictions and active trading behavior. Neither gives a calm, holistic view of where things actually stand. PortMira takes a different approach — a simple, local-first dashboard that helps users understand what they own and what they owe, clearly and privately.

PROBLEM STATEMENT

There is no lightweight, privacy-first tool that consolidates a user's assets and liabilities into one readable view without requiring cloud accounts, subscriptions, or opaque data sharing. Spreadsheets are flexible but demand manual formula work. Finance apps offer convenience but sacrifice privacy. Trading dashboards serve active investors, not users who simply want a calm, honest snapshot of their net worth. PortMira fills this gap.

TARGET USERS

PortMira is designed for individuals who hold assets across currencies and categories — cash in TWD or USD, US or Taiwan stocks, ETFs, and possibly crypto — and want a single, readable view of their position without relying on a commercial platform. The v0.1 user is someone comfortable entering their own data and running a Python application locally.

CORE PRODUCT PHILOSOPHY

Local-first by default

Privacy-conscious design

Clarity over prediction

Tracking, not advising

Calm over gamified

Working before ambitious

All user data stays in local files. Market prices are fetched from external APIs only as a read-only data source. PortMira will not give buy, sell, or hold recommendations. It will not show prediction charts or trading signals. The interface should feel analytical and trustworthy — not attention-driven or hype-oriented. AI features are deliberately out of scope for v0.1 and should only be considered after the core financial data layer is stable and accurate.

MVP FEATURES

Assets & liabilities

- Cash in multiple currencies (TWD, USD, EUR, JPY)
- Stocks and ETFs (US markets; Taiwan via `.TW / .TWO` if available)
- Crypto holdings
- Optional manually priced assets (bonds, commodities)
- Credit card debt, personal loans, margin loans

Dashboard & calculations

- Total assets, total liabilities, net worth
- Asset allocation by category (chart)
- Holdings table: value, cost basis, unrealized P/L, daily change
- Currency toggle: TWD ↔ USD
- Price movement alerts with user-defined thresholds (3%, 5%, 10%)
- Margin maintenance ratio calculator
- Loan / credit card interest calculator

Portfolio data is saved to and reloaded from a local JSON or CSV file, so users do not re-enter holdings each session.

DATA SOURCES AND APIS

PortMira uses only free, publicly accessible APIs as read-only data sources. Python and Pandas handle all cleaning, merging, and calculation — the app never displays raw API responses directly.

- **Stock and ETF prices:** `yfinance` or an equivalent free library, covering US tickers and Taiwan tickers where available.
- **Crypto prices:** CoinGecko public API or another free source such as CryptoCompare.
- **Foreign exchange rates:** A free FX API (e.g., ExchangeRate-API, Open Exchange Rates) or a static fallback rate table when the API is unavailable.

If any API call fails due to rate limits or network issues, the app falls back gracefully to the last cached price or a user-entered manual price. The application must remain usable without an active internet connection.

TECHNICAL ARCHITECTURE

The project separates financial logic from the interface layer, keeping modules independently testable and easier to migrate to other frameworks in the future.

```
portmira/
  app.py                # Main entry point (Streamlit or Flask)
  src/
    models.py           # Asset and liability data structures
    storage.py          # JSON / CSV load and save
    price_fetcher.py    # yfinance, crypto API, FX API, fallback logic
    calculations.py     # Net worth, P/L, allocation, alert system
    charts.py           # Matplotlib, Plotly, or Streamlit charts
  data/
    portfolio.json      # Local user portfolio data (sample included)
  docs/
    roadmap.md          # Future development plan
    architecture.md     # Architecture notes
  README.md
  requirements.txt
```

The UI will be built using **Streamlit or Flask** with HTML/CSS templates. Streamlit is preferred for rapid prototyping. Visualizations will use **Matplotlib, Plotly, or Streamlit's built-in chart components**. Pandas handles all data aggregation and calculation; raw API responses are never surfaced directly in the interface.

TEAM DIVISION OF LABOR

MEMBER	PRIMARY RESPONSIBILITY	KEY DELIVERABLES
Member A	Data & storage layer	models.py , storage.py , JSON/CSV schema, sample data file
Member B	Price fetching & APIs	price_fetcher.py , API integration, fallback logic, response caching
Member C	Calculations & alerts	calculations.py , net worth, P/L, allocation, alert system, calculators
Member D	UI & dashboard	app.py , charts.py , layout and navigation, README, demo prep

Each member owns a distinct module with a clear interface, minimizing merge conflicts. Integration sessions are scheduled at the end of Week 2 and Week 4.

DEVELOPMENT TIMELINE

- Week 1** Finalize data schema and asset/liability models. Set up repository. Confirm API choices and assign module ownership. Write sample portfolio data.
- Week 2** Build storage layer (JSON/CSV read/write). Implement price fetcher with fallback logic. Begin core calculations: net worth, allocation, unrealized P/L.
- Week 3** Integration checkpoint: connect storage, price fetcher, and calculations. Begin UI layout using sample data. Build alert system and basic calculators.
- Week 4** Complete dashboard: holdings table, allocation chart, currency toggle, alert display. Add currency conversion. Internal testing with real-world sample scenarios.
- Week 5** Polish UI, write README and documentation, prepare demo with sample portfolio, rehearse final presentation. Buffer time for bug fixes and edge cases.

RISKS AND FALLBACK PLANS

- **API failures or rate limits:** The app caches the last successful response and allows manual price entry. Core functionality works without internet.
- **Taiwan stock data gaps:** `yfinance` may not cover all Taiwan-listed tickers reliably. Manual price entry is a first-class fallback, not an afterthought.
- **Scope creep:** v0.1 is strictly bounded to local storage, price fetching, dashboard display, alerts, and two calculators. Any new feature goes to the roadmap.
- **Visualization complexity:** Charts start simple — a pie or bar chart for allocation, a plain table for holdings. Interactivity is added only if time allows.
- **Time pressure:** If the UI falls behind schedule, the team will demonstrate calculations through a simpler interface or Jupyter notebook rather than delay the working demo.

FUTURE ROADMAP

The following are explicitly **not** part of v0.1 but represent the natural next phases if the project continues as a public GitHub repository or eventual product:

- Repository cleanup, screenshots, polished README, and a public demo mode with sanitized sample data.
- Cached API responses and smarter refresh logic to reduce external requests.
- Additional asset classes: bonds, commodities, REITs, private holdings.
- Import from brokerage-exported CSV files (e.g., Interactive Brokers, Futu).
- Desktop application packaging or a mobile-friendly interface.
- Optional AI features — *only after the core data layer is stable* — such as plain-language portfolio summaries, natural language queries over local data ("What is my USD exposure?"), scenario analysis ("What if this holding drops 10%?"), and monthly financial reflection prompts. These act as an explanation layer, not an investment advisor.

WHY THIS PROJECT IS SUITABLE FOR A PYTHON FINAL PROJECT

PortMira covers the core Python curriculum naturally and purposefully. It uses dictionaries and lists for portfolio data, functions for calculation modules, and file I/O for local persistence — foundational concepts taught throughout the course. API integration introduces real-world data fetching. Pandas enables clean numerical processing and aggregation. The modular structure means four teammates can work in parallel on separate files with minimal merge conflicts.

The scope is deliberately bounded: the goal is a working, demonstrable local dashboard, not a production finance platform. The team can show a live demo with real prices and a sample portfolio, which makes for a clear and compelling final presentation. At the same time, the architecture is clean enough that the project can continue growing as a GitHub side project after the course ends — making the work genuinely useful, not just academically performative.